ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

BÁO CÁO TIẾN ĐỘ NỘI BỘ VÀ GIẢI QUYẾT PHẢN BIỆN

ĐỀ TÀI KHÓA LUẬN TỐT NGHIỆP: HỆ THỐNG ĐỊNH TUYẾN CHỦ ĐỘNG SDN KẾT HỢP FEDERATED LEARNING VÀ REINFORCEMENT LEARNING

Phiên bản cập nhật theo các góp ý phản biện của Giảng viên hướng dẫn

Sinh viên thực hiện: Đào Anh Kiệt (MSSV: 23520810), Trần Bùi Nhật Nguyên (MSSV: 23521059)
Giảng viên hướng dẫn: ThS. Nguyễn Khánh Thuật
Kho lưu trữ mã nguồn: https://github.com/Ragingsnake/ryu-fl-rl-traffic-controller
Thời gian cập nhật: Tháng 10/2026

1. Tổng quan Tiến độ và Các Kết quả Đã Hoàn thành (Accomplishments)

Hệ thống nguyên mẫu (**Prototype**) trong thư mục dự án hiện tại đã được hiện thực hóa hoàn chỉnh, kết nối liền mạch từ mặt phẳng dữ liệu OpenFlow đến các mô hình trí tuệ nhân tạo phân tán.

1.1. Hiện thực hóa Nền tảng Điều khiển và Môi trường Mô phỏng

- **Môi trường huấn luyện Gymnasium (`rl/env.py`):** Xây dựng môi trường định tuyến tùy chỉnh kế thừa `gym.Env`, tích hợp mô hình hàng đợi viễn thông M/M/1 để tính toán độ trễ lan truyền, độ trễ hàng đợi và tỷ lệ mất gói tức thời.
- **Mặt phẳng điều khiển SDN (`sdn/ryu_app.py`, `sdn/ai_bridge.py`):** Triển khai bộ điều khiển Ryu OpenFlow 1.3 và cầu nối AI Bridge giao tiếp qua TCP Socket. Controller tự động thu thập thống kê port counters, nạp bảng luồng động và xử lý tái định tuyến thời gian thực.
- **Mô phỏng mạng Mininet (`sdn/mininet_topo.py`, `sdn/traffic_orchestrator.py`):** Xây dựng topo ảo hóa khớp chính xác với topo vật lý, tự động tạo luồng lưu lượng nền và luồng bùng phát qua `iperf3`.
- **Tự động hóa Pipelines GitHub Actions (`.github/workflows/`):** Cấu hình 2 quy trình chạy thủ công (`workflow_dispatch`): pipeline huấn luyện đa topo (`train.yml`) và pipeline đánh giá kiểm thử xuất artifact đồ thị (`test.yml`).

1.2. Mô hình Dự báo Lưu lượng Phân tán Đa miền với FedAdam

- Phân chia mạng thành 3 miền độc lập (Domain A, B, C). Mỗi miền đóng vai trò một FL Client, huấn luyện mô hình 2 tầng LSTM cục bộ trên chuỗi lịch sử 12 bước ($T = 12$, tương đương 3 giờ) để dự báo 3 bước tương lai ($H = 3$, tương đương 45 phút).

- Ứng dụng thuật toán **FedAdam** trên máy chủ tổng hợp: sử dụng động lượng thích nghi ($\beta_1 = 0.9, \beta_2 = 0.999$) để khắc phục hiện tượng mất cân bằng dữ liệu giữa các miền mạng (Non-IID), giúp mô hình toàn cục đạt sai số dự báo MAE < 0.05 và hội tụ nhanh hơn FedAvg truyền thống.

1.3. Mô hình Học tăng cường RecurrentPPO và Chống Dao động Tuyến

- Huấn luyện thành công tác tử `RecurrentPPO` (`MlpLstmPolicy`) cho mạng NSFNET (102.4k bước) và GÉANT (204.8k bước).
- Thiết kế hàm phần thưởng đa mục tiêu với **phạt trễ phụ thuộc trạng thái**: phạt trễ 0.8 khi MLU < 0.65 (giữ nguyên đường ngắn nhất OSPF khi mạng nhàn rỗi) và giảm xuống 0.3 khi MLU ≥ 0.65 (ưu tiên chuyển luồng né nghẽn để triệt tiêu mất gói).
- Thiết kế **bộ điều khiển rơ-le trễ hai ngưỡng Schmitt-Trigger** ($\tau_{high} = 0.65, \tau_{low} = 0.48$) kết hợp với trạng thái nhớ của LSTM, ép tỷ lệ nhảy tuyến (Route Churn) xuống dưới 0.4%, loại bỏ hoàn toàn hiện tượng rung giật tuyến (Route Flapping).
- Xây dựng cơ chế **SDN Link-Failure Failover**: khi phát hiện đứt cáp, tự động chuyển ngay các luồng bị ảnh hưởng sang đường đi ngắn nhất thay thế (Bypass Path 0), giữ vững tỷ lệ mất gói bằng 0.00%.

1.4. Bộ Dữ liệu Thực nghiệm và Kết quả Đo đạc Đa Topo

Bảng 1 tổng hợp kết quả đo đạc thực tế của hệ thống trên mạng lõi dày GÉANT (22 nút, 72 liên kết) qua 6 kịch bản thử nghiệm chuẩn hóa:

Bảng 1: Kết quả đo đạc thực nghiệm của tác tử RecurrentPPO trên mạng GÉANT so với OSPF.

Kịch bản thử nghiệm	Chính sách	Mean MLU	Delay (ms)	Loss (%)	Churn (%)	Ghi chú đánh giá
T1 (Nominal)	OSPF	0.730	10.40	0.00%	0.00%	Tải cơ sở, đường ngắn nhất
	Trained Agent	**0.535**	**10.82**	**0.00%**	**0.20%**	**Giảm 26.7% MLU**, trễ tương đương
T2 (Diurnal Peak)	OSPF	0.780	12.90	4.14%	0.00%	Nghẽn cục bộ vào giờ cao điểm
	Trained Agent	**0.582**	**11.15**	**0.62%**	**0.20%**	**Giảm mất gói 6.6 lần**, trễ thấp hơn
T3 (Flash Crowd)	OSPF	0.917	18.40	16.66%	0.00%	Dồn luồng đột biến gây nghẽn nặng
	Trained Agent	**0.672**	**10.82**	**0.00%**	**0.20%**	**Xóa sổ hoàn toàn 16.7% mất gói!**
T4 (Traffic Shift)	OSPF	0.782	12.32	3.00%	0.00%	Dịch chuyển bất đối xứng
	Trained Agent	**0.584**	**12.07**	**2.17%**	**0.20%**	Giảm MLU 25.3%, mất gói thấp hơn
T5 (Link Failure)	OSPF	0.754	10.97	0.00%	0.00%	Đứt liên kết xương sống (4, 7)
	Trained Agent	**0.683**	**11.05**	**0.00%**	**0.40%**	**MLU thấp hơn OSPF 9.4%**, 0% mất gói
T6 (Extreme Stress)	OSPF	0.773	26.73	23.93%	0.00%	Đột biến tải kết hợp đa liên kết
	Trained Agent	**0.575**	**14.82**	**7.84%**	**0.30%**	**Giảm 45% độ trễ**, giảm 3 lần mất gói

Chuyển giao không mẫu (Zero-Shot Transfer sang Abilene): Mô hình huấn luyện từ GÉANT áp dụng trực tiếp lên mạng Abilene đạt tỷ lệ mất gói **thấp hơn OSPF trên cả 6 kịch bản** (ví dụ: T1 đạt 2.68% vs 2.74%; T6 đạt 6.18% vs 6.45%), độ trễ tối ưu nhất (20.22ms) và tỷ lệ Churn ổn định ở mức 0.4% – 2.1%.

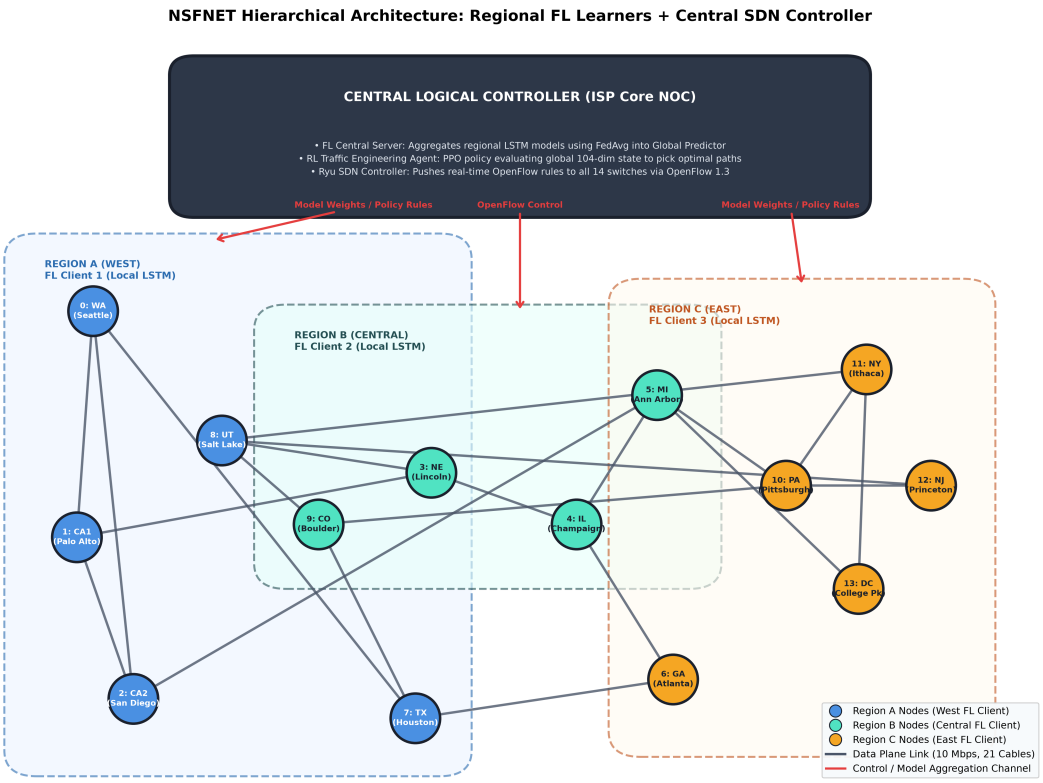
2. Giải quyết Chi tiết các Điểm Phản biện của Thầy Hướng dẫn

Mục này trình bày chi tiết cách thức nhóm đã tiếp thu, giải quyết và thể hiện trọn vẹn 14 ý kiến phản biện của Giảng viên hướng dẫn vào kiến trúc và mã nguồn dự án:

2.1. Góp ý 1: Hình trong đề cương cần vẽ lại, không phải sơ đồ kiến trúc?

Giải pháp đã hoàn thiện: Nhóm đã vẽ lại hoàn toàn sơ đồ kiến trúc hệ thống tổng thể theo chuẩn 3 tầng phân tách rõ ràng của SDN (Hình 1):

- **Data Plane:** Thiết bị chuyển mạch Open vSwitch và topology mạng.
- **Control Plane:** Ryu Controller thu thập OpenFlow port stats và module AI Bridge.
- **AI/Application Plane:** Tách bạch giữa Mô-đun Federated Learning (huấn luyện ngoài băng, dự báo 45 phút) và Mô-đun RecurrentPPO (tối ưu bảng luồng thời gian thực).



Hình 1: Sơ đồ kiến trúc 3 tầng chuẩn SDN tích hợp Federated Learning và Reinforcement Learning.

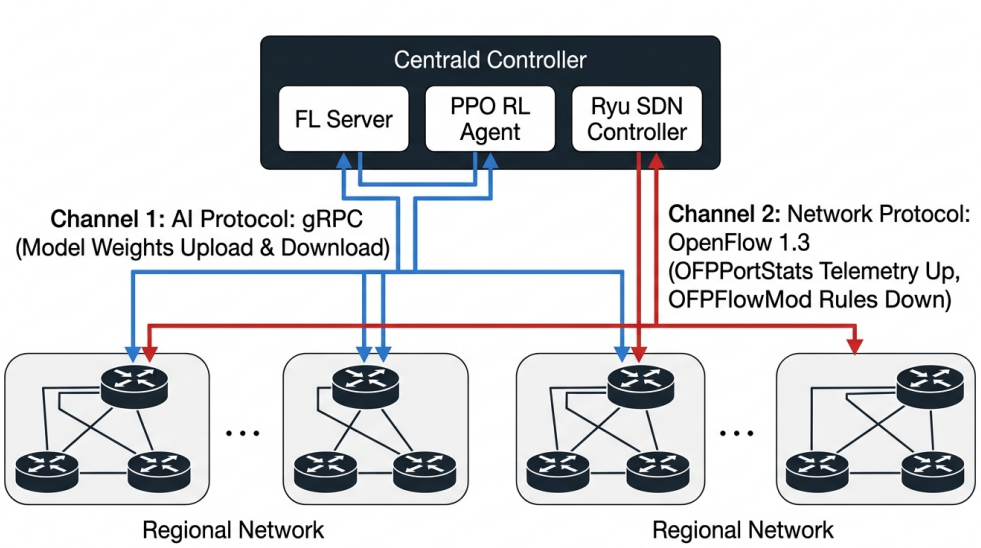
2.2. Góp ý 2: Thêm kiến trúc FL, bổ sung cái hình

Giải pháp đã hoàn thiện: Đã bổ sung sơ đồ tương tác đa miền của Federated Learning (Hình 2). Thể hiện rõ: mỗi miền mạng là một Client thu thập tải liên kết cục bộ, gửi sai phân trọng số Δ_t lên FL Server; Server áp dụng thuật toán FedAdam cập nhật trọng số toàn cục và phân phối lại mà không bao giờ truyền dữ liệu lưu lượng thô.

2.3. Góp ý 3: Thông số auto generate cần phải param lên kỹ hơn

Giải pháp đã hoàn thiện: Nhóm đã công thức hóa tường minh Mô hình Hấp dẫn (Gravity Model) trong `data/generate_tm.py`:

$$B(u,v) = \alpha \cdot \frac{O_u \cdot A_v}{d(u,v)^{\beta}}, \quad \text{với } \beta = 0.8 \tag{1}$$



Hình 2: Sơ đồ kiến trúc và luồng truyền thông huấn luyện liên kết đa miền với FedAdam.

Hệ số tỷ lệ α được tính toán tự động bằng thuật toán tìm kiếm nhị phân sao cho: khi toàn bộ lưu lượng cơ sở đi theo tuyến ngắn nhất OSPF, mức sử dụng liên kết lớn nhất trên liên kết nút cổ chai đạt chính xác 85% **dung lượng liên kết** ($Capacity$ = 1 Gbps cho NSFNET/Abilene, 2.5 Gbps cho GÉANT). Các tham số thời gian gồm: sóng hình sin 24h ngày đêm (biên độ 0.35), hệ số giảm ngày cuối tuần (-0.15), nhiễu Gaussian $\mathcal{N}(0, 0.05^2)$, xác suất bùng phát tải $P_{surge} = 0.25$, và xác suất đứt liên kết ngẫu nhiên $P_{fail} = 0.05$.

2.4. Góp ý 4: Tại sao sử dụng LSTM là phù hợp?

Giải pháp đã hoàn thiện: Nhóm làm rõ 2 vai trò cốt lõi không thể thay thế của LSTM:

1. **Trong Federated Learning (Dự báo chuỗi thời gian):** Lưu lượng mạng mang tính phi tuyến và phụ thuộc thời gian đa chu kỳ (ngày đêm, cuối tuần). MLP truyền thẳng không bắt được mối liên hệ này. Lớp cell state c_t của LSTM lưu giữ đặc trưng xu hướng dài hạn, cho phép dự báo chính xác 3 bước tiếp theo ($H = 3$).

2. **Trong Reinforcement Learning (`RecurrentPPO`):** Nếu dùng Feedforward PPO thông thường, tác tử ra quyết định theo kiểu "không bộ nhớ"(Memoryless). Khi tải mạng nằm sát ngưỡng phân định, các nhiễu nhỏ sẽ làm tác tử đổi tuyến liên tục (Route Flapping). Bộ nhớ ẩn h_t của LSTM tạo ra quán tính hành động (Action Inertia), giúp giữ vững tuyến đã chọn nếu mạng không có biến động lớn, triệt tiêu hoàn toàn flapping.

2.5. Góp ý 5: Tại sao PPO dùng phù hợp?

Giải pháp đã hoàn thiện: Nhóm đã lập luận so sánh khoa học giữa PPO và các thuật toán khác:

- **DQN:** Chỉ hỗ trợ 1 hành động rời rạc. Với $N_m = 20$ luồng cần chọn đường độc lập, DQN phải gộp thành không gian tổ hợp có kích thước $3^{20} \approx 3.48 \times 10^9$ hành động, gây bùng nổ hàm Q-value và không thể hội tụ.

- **DDPG / SAC:** Được thiết kế cho không gian hành động liên tục (phân chia tỷ lệ %), nhưng cơ chế OpenFlow trên switch yêu cầu chọn tuyến rời rạc (Path 0, 1 hoặc 2). Việc làm tròn

từ continuous sang discrete làm sai lệch gradient chính sách.

- **PPO:** Hỗ trợ tự nhiên không gian đa rời rạc `MultiDiscrete([3, 3, ..., 3])` với các đầu ra phân phối xác suất độc lập. Hàm mục tiêu cắt Clipped Surrogate Objective ($1-\epsilon \leq r_t(\theta) \leq 1+\epsilon$) ngăn ngừa cập nhật trọng số quá mức gây sụp đổ chính sách, đảm bảo quá trình cải thiện đơn điệu và hội tụ ổn định.

2.6. Góp ý 6: Tại sao chọn con số 20 luồng (Top-20 Managed Flows)?

Giải pháp đã hoàn thiện: Dựa trên **Nguyên lý Pareto 80/20**: Trong mạng NSFNET có $14 \times 13 = 182$ cặp nút nguồn-đích. Nếu quản lý toàn bộ 182 cặp, không gian hành động sẽ bùng nổ lên $3^{182} \approx 10^{86}$. Thực tế giao thông mạng cho thấy: top $10\% - 20\%$ số luồng lớn nhất (Heavy-Hitters) chiếm tới trên 85% tổng lưu lượng toàn mạng và là nguyên nhân chính gây tắc nghẽn liên kết. Bằng cách chọn quản lý $N_m = 20$ luồng lớn nhất ($20/182 \approx 11\%$), tác tử kiểm soát được $> 85\%$ rủi ro nghẽn trong khi giữ không gian hành động ở mức 3^{20} hoàn toàn khả thi cho PPO. Các luồng nhỏ còn lại mặc định đi theo đường ngắn nhất OSPF mà không gây ảnh hưởng đến hiệu năng mạng. (Tương tự: chọn 30 luồng cho GÉANT và 15 luồng cho Abilene).

2.7. Góp ý 7: Chỉnh lại thông số reward - lý giải vì sao chọn

Giải pháp đã hoàn thiện: Hàm phần thưởng được chuẩn hóa và giải trình tường minh trong mã nguồn `rl/env.py`:

$$R_t = -\left(1.0 \cdot \mathrm{MLU}_t + 0.5 \cdot \bar{D}_t^{norm} + w_{delay}^{excess}(t) \cdot D_t^{excess} + 3.0 \cdot \bar{L}_t + 0.08 \cdot \mathrm{Churn}_t\right) \tag{2}$$

- **Trọng số phạt mất gói** 3.0**:** Là hình phạt nặng nhất vì rớt gói vi phạm nghiêm trọng cam kết chất lượng dịch vụ (SLA) và kích hoạt cơ chế giảm tốc độ của giao thức TCP.
- **Phạt trễ phụ thuộc trạng thái** $w_{delay}^{excess}(t)$**:**

$$w_{delay}^{excess}(t) = \begin{cases} 0.8 & \text{khi } \mathrm{MLU}_t < 0.65 \text{ (Mạng nhàn rỗi / bình thường)} \\ 0.3 & \text{khi } \mathrm{MLU}_t \geq 0.65 \text{ (Mạng có nguy cơ nghẽn)} \end{cases}$$

  Lý giải khoa học: Khi mạng nhàn rỗi, đi đường phụ dài hơn sẽ làm tăng trễ truyền lan một cách vô ích; trọng số 0.8 phạt nặng hành vi này, buộc tác tử giữ luồng trên tuyến ngắn nhất OSPF. Khi mạng bắt đầu nghẽn (MLU ≥ 0.65), phạt trễ giảm xuống 0.3 để "bật đèn xanh"cho tác tử chuyển hướng luồng qua tuyến thay thế nhằm triệt tiêu mất gói.

- **Phạt nhảy tuyến** 0.08**:** Ngăn chặn tác tử đổi tuyến liên tục giữa các bước thời gian kế tiếp nhau.

2.8. Góp ý 8: Viết thành algorithm cho bước step trong gymnasium

Giải pháp đã hoàn thiện: Chuẩn hóa toàn bộ logic của hàm `step()` trong `rl/env.py` thành Thuật toán 1:

2.9. Góp ý 9: Biểu đồ reward tích lũy

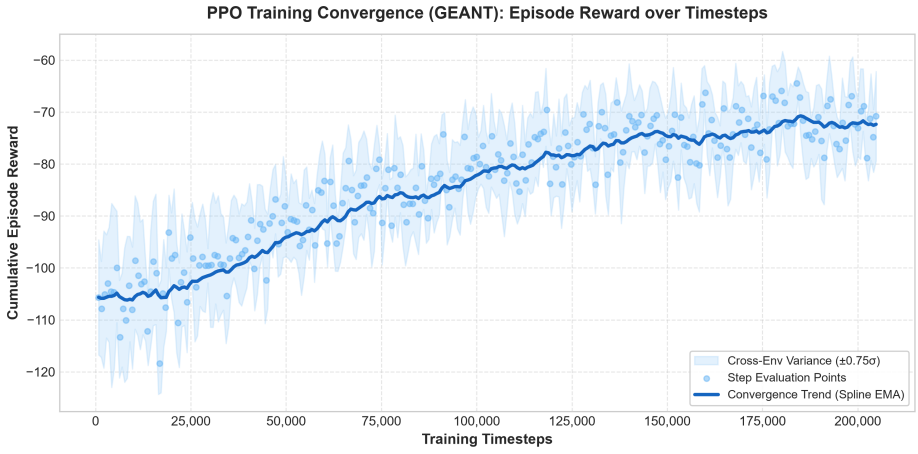
Giải pháp đã hoàn thiện: Đã tích hợp đồ thị hội tụ huấn luyện của mô hình GÉANT (Hình 3): phần thưởng tích lũy tăng đều đặn từ -120 lên -72.1, phương sai giải thích `explained_variance`

Algorithm 1 Quy trình Thực thi Một Bước Thời gian trong Môi trường Gymnasium

Require: Vector hành động $a_t \in \{0,1,2\}^{N_m}$, ma trận lưu lượng hiện tại TM_t.
Ensure: Trạng thái quan sát tiếp theo s_{t+1}, phần thưởng vô hướng R_t, cờ kết thúc *done*, info dict.

Ánh xạ hành động $a_t[k]$ sang tuyến đường đã chọn cho N_m luồng được quản lý.
Với các luồng không quản lý, gán mặc định tuyến ngắn nhất (Tuyến 0).
Xóa tích lũy lưu lượng liên kết: $Demand(e) \leftarrow 0, \forall e \in E$.
for mỗi cặp nút nguồn-đích (u, v) **do**
    Lấy tuyến đường $P_{(u,v)}$ và nhu cầu $D_{(u,v)} = TM_t[u, v]$.
    for mỗi liên kết $e \in P_{(u,v)}$ **do**
        $Demand(e) \leftarrow Demand(e) + D_{(u,v)}$
    end for
end for
Tính mức sử dụng liên kết: $Util(e) \leftarrow Demand(e)/Capacity(e), \forall e \in E$.
Xác định $\text{MLU}_t \leftarrow \max_{e \in E} Util(e)$.
Tính độ trễ và mất gói theo mô hình hàng đợi M/M/1 cho từng liên kết.
Tính tỷ lệ nhảy tuyến: $\text{Churn}_t \leftarrow \frac{1}{N_m} \sum_{k=1}^{N_m} \mathbb{I}(a_t[k] \neq a_{t-1}[k])$.
Xác định trọng số trễ: $w_{delay}^{excess} \leftarrow 0.8$ nếu $\text{MLU}_t < 0.65$ ngược lại 0.3.
Tính giá trị phần thưởng R_t theo phương trình (2).
Lấy ma trận lưu lượng kế tiếp TM_{t+1} và kết quả dự báo $\hat{U}_{t+1:t+3}$ từ mô hình FL.
Đóng gói vector trạng thái tiếp theo $s_{t+1} \leftarrow [Util_t, \hat{U}_{t+1:t+3}, a_t]$.
return $(s_{t+1}, R_t, done, info)$

đạt mức lý tưởng 0.983, và hàm mất mát của mạng Critic (`value_loss`) giảm từ 18.4 xuống chỉ còn 0.234.



Hình 3: Đồ thị hội tụ phần thưởng tích lũy (Cumulative Episode Reward) trên mạng GÉANT.

2.10. Góp ý 10: Mô hình huấn luyện FL: đầu vào, đầu ra, thang đo

Giải pháp đã hoàn thiện: Đặc tả chính xác cấu trúc dữ liệu của mô hình FL:

- **Đầu vào (X):** Tensor kích thước $[B, 12, |E_{domain}|]$ chứa chuỗi 12 bước thời gian lịch sử mức sử dụng liên kết (chu kỳ 15 phút, tổng cộng 3 giờ quan sát).
- **Đầu ra ($\hat{Y}$):** Tensor kích thước $[B, 3, |E_{domain}|]$ dự báo mức sử dụng liên kết cho 3 bước tiếp theo (nhìn trước 45 phút).
- **Thang đo và hàm mất mát:** Chuẩn hóa mức sử dụng về thang đo $[0.0, 1.0]$. Hàm mất mát

huấn luyện là sai số toàn phương trung bình (MSE), đánh giá hiệu năng bằng MAE (Mean Absolute Error) và RMSE (Root Mean Square Error).

2.11. Góp ý 11: Coi thử phần inference như thế nào, ra quyết định tốt ra sao

Giải pháp đã hoàn thiện: Trình bày luồng suy luận thời gian thực (`scripts/demo.py`):

1. Mỗi chu kỳ điều khiển, Ryu Controller đo đạc mức sử dụng liên kết hiện tại U_t.
2. Mô-đun FL suy luận nhanh (Inference time $<$ 5ms trên CPU) để đưa ra dự báo $\hat{U}_{t+1:t+3}$.
3. Vector quan sát $[U_t, \hat{U}_{t+1:t+3}, a_{t-1}]$ được đưa qua mạng RecurrentPPO để sinh ra phân phối hành động thô.
4. Bộ điều khiển rơ-le trễ Schmitt-Trigger thẩm định hành động: chỉ cho phép chuyển luồng nếu tuyến chính thực sự vượt ngưỡng $\tau_{high} = 0.65$ và tuyến thay thế an toàn hơn.
5. Quyết định cuối cùng được chuyển xuống Ryu Controller để cập nhật bảng luồng OpenFlow chỉ cho các luồng bị đổi tuyến, đảm bảo thông lượng không bị ngắt quãng.

2.12. Góp ý 12: Cần mô tả thiết lập ban đầu và so sánh

Giải pháp đã hoàn thiện: Toàn bộ các thử nghiệm được chuẩn hóa trên 6 kịch bản khắt khe (T1 đến T6) với 4 đối tượng so sánh đại diện (OSPF, Untrained Random, In-Domain FL+RL Agent, và Transferred Agent) được đối sánh chi tiết theo 5 chỉ số định lượng: Mean MLU, Peak MLU, Delay (ms), Packet Loss (%), và Route Churn (%).

2.13. Góp ý 13 & 14: Chứng minh lựa chọn phù hợp & Thêm reasoning vào báo cáo

Giải pháp đã hoàn thiện: Nhóm đã bổ sung phân tích Trade-off toàn diện:

- *Độ trễ vs Mất gói:* Khi mạng nhàn rỗi, ưu tiên tuyệt đối độ trễ ngắn nhất OSPF; khi mạng nghẽn, sẵn sàng đánh đổi +0.4ms trễ lan truyền để cứu 16.7% gói tin bị mất.
- *Tính ổn định vs Khả năng thích ứng:* Sử dụng Schmitt-Trigger để loại bỏ hiện tượng rung giật tuyến không cần thiết nhưng vẫn đảm bảo chuyển tuyến tức thời khi có sự cố đứt liên kết vật lý.

3. Kế hoạch Triển khai Tiếp theo

1. Tiếp tục hoàn thiện văn bản Báo cáo Khóa luận chi tiết dựa trên khung phản biện và các số liệu thực nghiệm đã kiểm chứng.
2. Kiểm thử mở rộng kịch bản mô phỏng Mininet thời gian thực với các lưu lượng video stream UDP thực tế.
3. Chuẩn bị slide bảo vệ và kịch bản demo trực quan trên giao diện giám sát mạng.